A Report

on

LEGALBERT BASED MULTI-LABEL IPC SECTION RECOMMENDATION ENGINE

carried out as part of the course Project Based Learning- 2/4

Submitted by

PankhudiRaina 2427030730 IV Semester

in partial fulfilment for the award of the degree

of

BACHELOR OF TECHNOLOGY

In

Computer Science & Engineering

Department of Computer Science & Engineering,
School of Computer Science and Engineering,
Manipal University Jaipur,
Jan-May 2026

A Report

on

LEGALBERT BASED MULTI-LABEL IPC SECTION RECOMMENDATION ENGINE

carried out as part of the course Project Based Learning-2/4

Submitted by

PankhudiRaina 2427030730 IV Semester

in partial fulfilment for the award of the degree

of

BACHELOR OF TECHNOLOGY

In

Computer Science & Engineering

Under the Guidance of :

Guide Name : Ms. Neha

Guide Signature(with date) : ……

Acknowledgement

This project would not have been completed without the help, support, comments, advice, cooperation and coordination of various people. However, it is impossible to thank everyone individually; I am hereby making a humble effort to thank some of them.

I acknowledge and express my deepest sense of gratitude to my internal supervisor **Ms. Neha** for his/her constant support, guidance, and continuous engagement. I highly appreciate his technical comments, suggestions, and criticism during the progress of this project titles "**LegalBert based Multi-Label IPC Section Recommendation Engine**".

I owe my profound gratitude to ***Dr. Neha Chaudhary*** , Head, Department of CSE, for her valuable guidance and for facilitating me during my work. I am also very grateful to all the faculty members and staff for their precious support and cooperation during the development of this project.

Finally, I extend my heartfelt appreciation to my classmates for their help and encouragement.

Pankhudi Raina

2427030730

MANIPAL UNIVERSITY
JAIPUR
(University under Section 2(f) of the UGC Act)

Department of Computer Science and Engineering

School of Computer Science and Engineering

Date: _____

CERTIFICATE

This is to certify that the project entitled "***LegalBert based Multi-Label IPC Section Recommendation Engine"*** is a bonafide work carried out as PBL-2 ***End Term Assessment*** in partial fulfillment for the award of the degree of Bachelor of Technology in Computer Science and Engineering, by ***Pankhudi Raina*** bearing registration number **2427030730**, during the academic semester IV *of year 2025-2026.*

Place: Manipal University Jaipur, Jaipur

Name of the project guide: Ms. Neha

Signature of the project guide: _____

Abstract

The identification of applicable Indian Penal Code (IPC) sections from legal case descriptions is a cognitively intensive task that demands substantial domain expertise. Manual assignment of IPC sections is time-consuming, inconsistent, and prone to error in overburdened judicial systems. This project presents an automated multi-label classification system leveraging LegalBERT, a domain-adapted transformer model pre-trained on diverse legal corpora to predict relevant IPC sections directly from natural language legal text, thereby supporting legal practitioners in efficient statute identification [1].

The system is trained on the Indian Legal Statute Identification (ILSI) dataset comprising 42,835 annotated samples across 116 IPC section classes. Training employs Binary CrossEntropy with Logits Loss, the AdamW optimiser with warm-up scheduling, fp16 mixed precision, and gradient checkpointing across 3 epochs. A MultiLabelBinarizer encodes multilabel targets, and a sigmoid activation over the classification head produces independent persection probabilities.

An optimized global threshold of 0.3102 derived by maximizing F1-Micro on the development set governs binary classification decisions. On the ILSI test set, the system achieves F1-Micro 0.5051, Precision 0.8302, Recall 0.3629 at the default threshold. Cross-domain evaluation on the unseen IndianBailJudgments, 1200 dataset yields F1-Micro 0.4234, demonstrating meaningful generalization. Gradient-based token importance provides interpretable explanations for each prediction.

Table of Contents

Acknowledgement i
Abstract ii
1. Introduction 1
    1.1 Objective of the Project 1
    1.2 Brief Description 2
    1.3 Technology Used 3
    1.4 Organization Profile 4
2. Design Description 5
    2.1 Flowchart 5
    2.2 Data Flow Diagrams 7
    2.3 ER Diagram 9
3. Project Description 10
    3.1 Database 10
    3.2 Table Description 11
    3.3 File / Database Design 12
4. Input / Output Form Design 13
5. Testing and Tools Used 14
6. Implementation and Maintenance 16
7. Conclusion and Future Work 18
8. Outcome 19
9. Bibliography 20

1. Introduction

The Indian Penal Code, 1860 constitutes the foundational statute of Indian criminal law, enumerating offences ranging from petty theft to capital murder. In legal practice, accurate identification of applicable IPC sections is a prerequisite for preparing charge sheets, conducting bail hearings, and formulating legal arguments. Despite its importance, this task remains largely manual a process that is laborious, expert-dependent, and frequently inconsistent across jurisdictions [1].

The emergence of pre-trained transformer models in Natural Language Processing has opened new avenues for automating complex text understanding tasks in specialized domains. LegalBERT [2], introduced by Chalkidis et al., adapts the BERT pre-training paradigm to the legal domain by training on a large corpus of EU legislation, US court opinions, and legal contracts. The resulting model captures domain-specific vocabulary, citation patterns, and legal reasoning structures absent from general-purpose language models.

This project fine-tunes LegalBERT on the ILSI dataset for automated IPC section prediction an inherently multi-label classification problem, since a single legal description may warrant multiple concurrent IPC sections reflecting the doctrine of composite offences. The system further integrates gradient-based explainability to highlight the specific words driving each section prediction, adding transparency essential for legal deployment.

1.1 Objective of the Project

The objectives of this project are:

- To build an end-to-end automated pipeline that predicts applicable IPC sections from free-form natural language descriptions of criminal incidents and legal cases.
- To fine-tune LegalBERT (nlpaueb/legal-bert-base-uncased) for multi-label classification over 116 IPC section categories using the ILSI dataset via the HuggingFace Trainer API.
- To optimize the classification threshold on the development set by maximizing F1Micro, identifying the optimal threshold of 0.3102.
- To provide gradient-based explainability that identifies the most influential tokens for each predicted IPC section, making the system interpretable for legal practitioners.

- To evaluate cross-domain generalization on the IndianBailJudgments-1200 dataset without any retraining, and to justify the precision-recall tradeoff in the legal context.
- To produce a reproducible pipeline with modular components for data loading, training, evaluation, inference, and explanation.

1.2 Brief Description of the Project

The system implements a complete ML pipeline in five stages: data loading and preprocessing; model training; threshold optimization; inference and explainability; and cross-domain evaluation.

In preprocessing, raw JSONL files are parsed; IPC labels are extracted via regex pattern matching (r'Section\s+(\d+[A-Z]?)'); list-type text fields are normalized to single strings; and a MultiLabelBinarizer fitted on the training set encodes labels into 116-dimensional binary vectors.

The model backbone is loaded using AutoModelForSequenceClassification with problem_type='multi_label_classification', automatically enabling BCEWithLogitsLoss. The IPCDataset class tokenizes text on-the-fly (max_length=512). Training uses the HuggingFace Trainer with AdamW (lr=1e-5, weight_decay=0.01), 500 warmup steps, fp16, gradient checkpointing, and eval/save per epoch over 3 epochs.

The threshold optimization step evaluates F1-Micro on the development set for threshold candidates in the range [0.1, 0.9], selecting 0.3102 as the global optimum. The inference function additionally implements short-query expansion and a confidence-based fallback strategy. The explainability module computes gradient attribution on input embeddings. Crossdomain evaluation applies the trained model directly to IndianBailJudgments-1200 with identical threshold and no adaptation.

1.3 Technology Used

1.3.1 Hardware Requirements

Table 1: Hardware Requirements

Component	Specification
GPU	NVIDIA T4 (16 GB VRAM) — Google Colab

RAM	12.7 GB system RAM (Colab environment)
Storage	Google Drive: ~15 GB for model, dataset, and pickle
CPU	Intel Xeon @ 2.20 GHz (2 vCPUs)
Network	Stable internet for HuggingFace Hub access

1.3.2 Software Requirements

Table 2: Software Requirements

Software / Library	Version / Details
Python	3.10.x (Google Colab)
PyTorch	2.0+ with CUDA 11.8
HuggingFace Transformers	4.30+ (Trainer, AutoModel, AutoTokenizer)
HuggingFace Datasets	2.12+ (load_dataset)
Scikit-learn	1.2+ (MultiLabelBinarizer, metrics, multilabel_confusion_matrix)
NumPy	1.24+
Pandas	2.0+
Matplotlib / Seaborn	3.7+ (confusion matrix visualization)
Google Colab + Drive	Cloud training and checkpoint persistence

1.4 Organization Profile

Not Applicable.

2. Design Description

2.1 Flowchart

The system flowchart below depicts the complete end-to-end prediction pipeline from raw legal text input to the final output comprising predicted IPC sections, confidence scores, gradientbased token explanations, and a match analysis report. Nine sequential stages are

illustrated, including a decision diamond for threshold comparison and a fallback branch for lowconfidence cases.

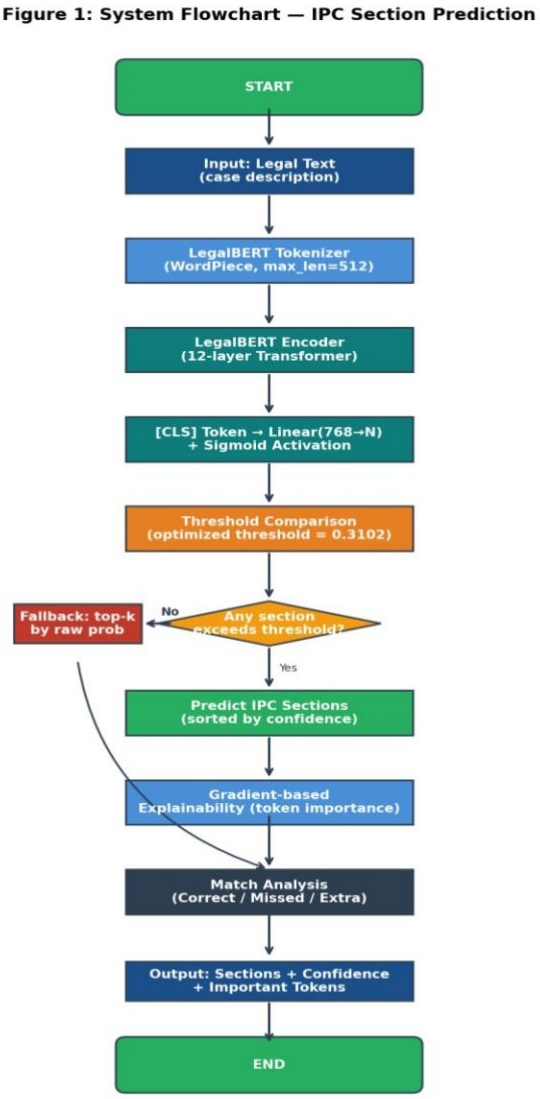

Figure 1: End-to-End System Flowchart — IPC Section Prediction

The system architecture diagram below presents the layer-by-layer data flow from the tokenized input through the LegalBERT encoder, dropout, linear classification head, and sigmoid activation to the final output tier. Each processing layer is color-coded by function: blue for input/encoding, teal for transformation, orange for regularization/thresholding, and green for output generation.

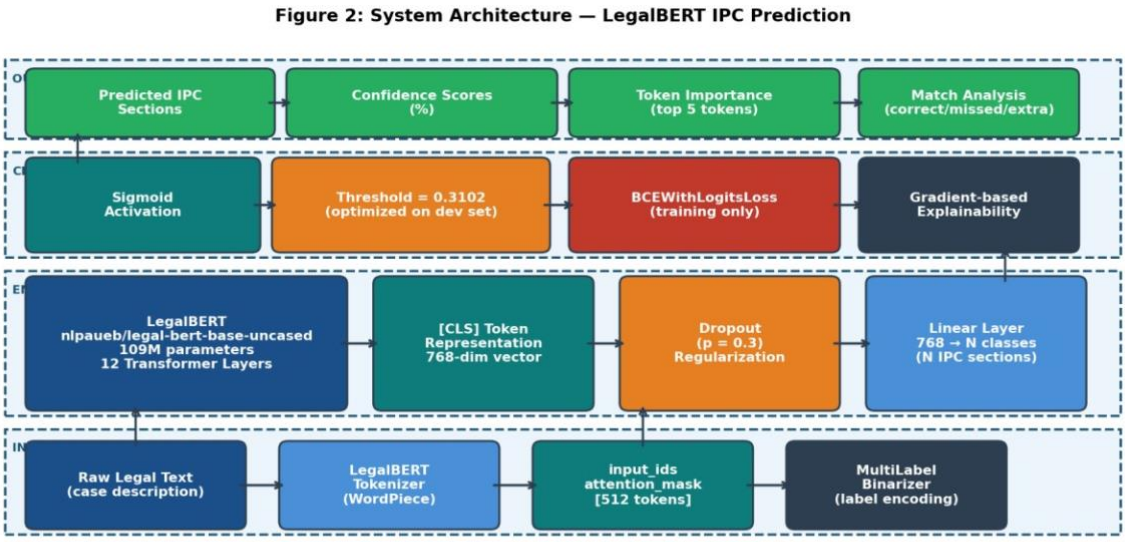

Figure 2: System Architecture — LegalBERT IPC Prediction Pipeline

2.2 Data Flow Diagrams (DFDs)

2.2.1 DFD Level 0 — Context Diagram

The Level 0 DFD represents the entire system as a single process node. Four external entities interact with it: (i) the Legal User, who supplies crime descriptions and receives predicted IPC sections; (ii) the ILSI Dataset, which provides annotated training records; (iii) the HuggingFace Model Hub, from which pre-trained LegalBERT weights are downloaded; and (iv) the IPC Knowledge Base, which supplies the section-name-to-label-index mapping used by the MultiLabelBinarizer.

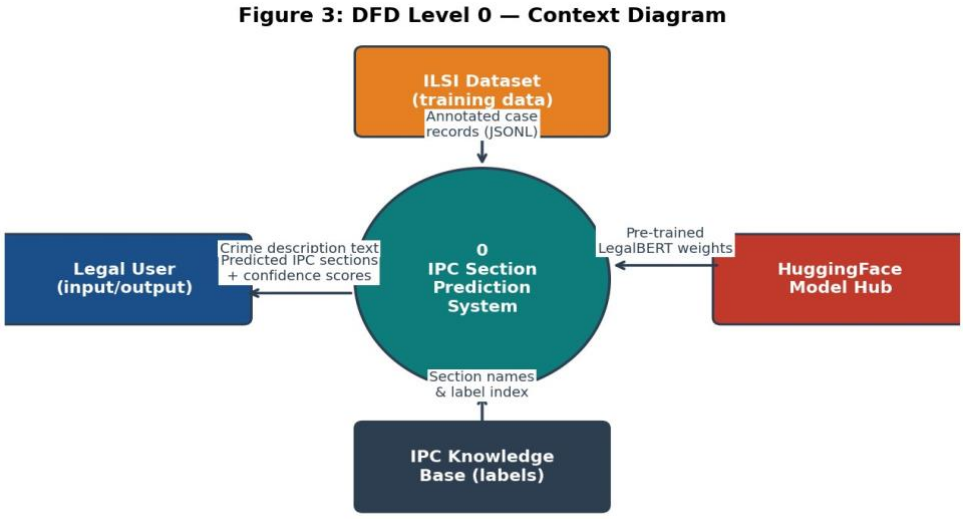

Figure 3: DFD Level 0 — Context Diagram

2.2.2 DFD Level 1 - Sub-process Decomposition

The Level 1 DFD decomposes the system into four major sub-processes. Process 1.0 (Data Ingestion & Preprocessing) reads JSONL files, extracts and normalizes labels, tokenizes text, and writes batches to the Training Data Store. Process 2.0 (Model Training) reads batches, executes forward/backward passes, and saves the best model checkpoint. Process 3.0 (Evaluation & Threshold Calibration) computes the optimal global threshold on the dev split and records it in the Threshold Store. Process 4.0 (Inference Engine) reads user text, the trained model, and the threshold to produce predictions and explanations.

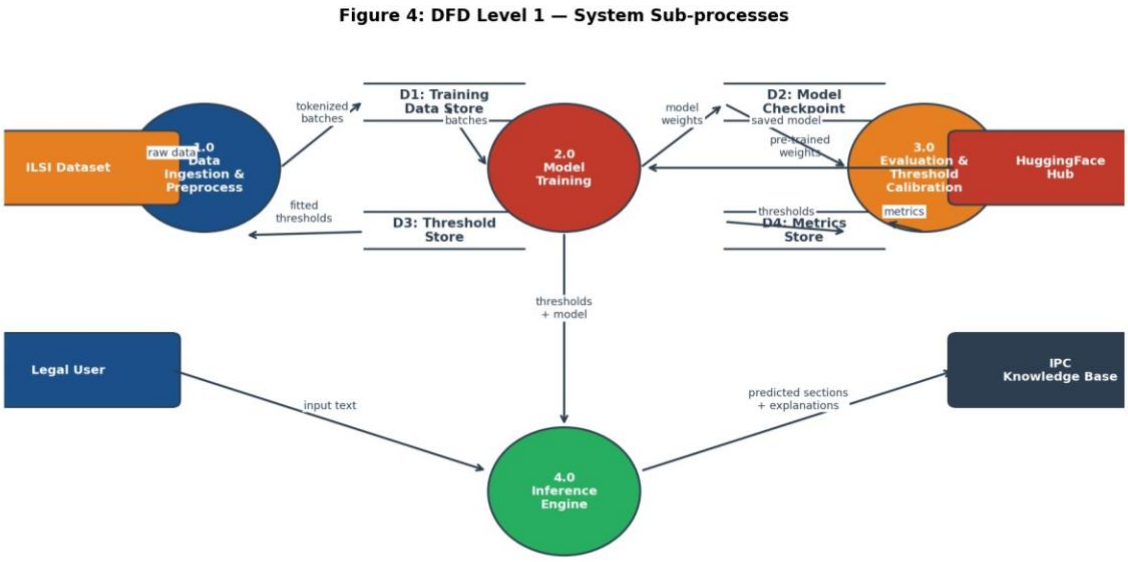

Figure 4: DFD Level 1 — System Sub-process Decomposition

2.2.3 DFD Level 2 - Training Sub-process

Within Process 2.0 (Model Training), the detailed operations are: (2.1) DataLoader fetches mini-batches of size 8; (2.2) Trainer executes the forward pass under fp16 AMP with gradient checkpointing enabled; (2.3) BCEWithLogitsLoss is computed against multi-hot encoded labels; (2.4) AdamW updates weights with warm-up scheduling; and (2.5) the Trainer saves the best checkpoint determined by validation loss at each epoch boundary, retaining at most two checkpoints.

2.3 Entity Relationship (ER) Diagram

The ER diagram below illustrates the five logical data entities of the system and their cardinality relationships. CASE stores individual training and test instances. IPC_SECTION represents the 116 distinct statutory provisions. ANNOTATION is the many-to-many junction entity recording which sections apply to which cases and whether each was predicted. MODEL represents the saved fine-tuned checkpoint. THRESHOLD records per-model classification thresholds, one per section.

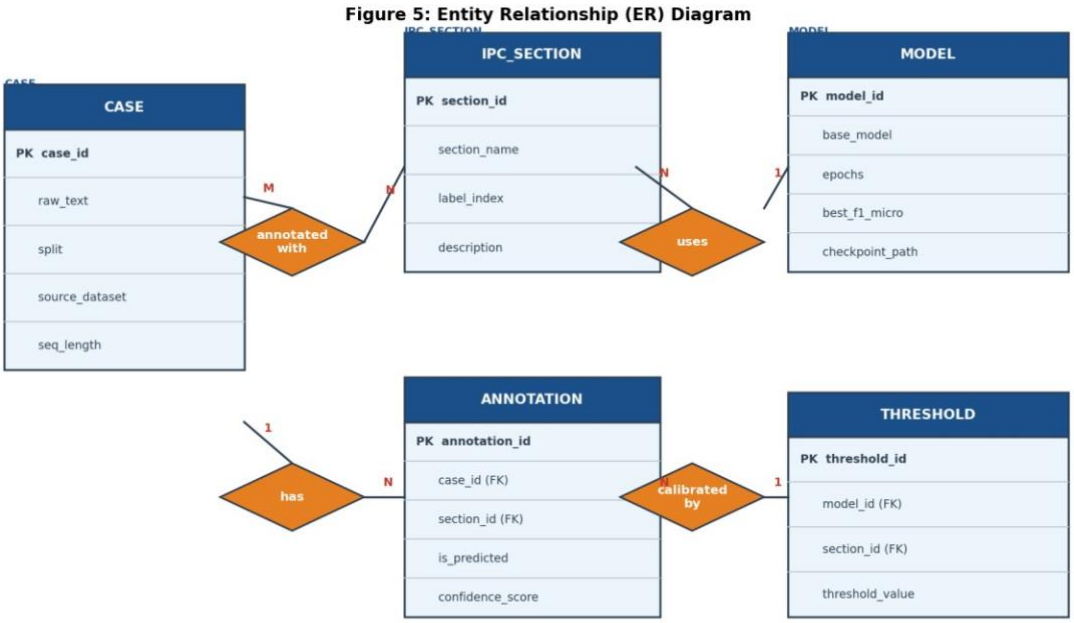

Figure 5: Entity Relationship Diagram — System Data Model

3. Project Description

3.1 Database

The system does not employ a live relational DBMS. Data is stored through file-based formats persisted on Google Drive: JSONL files for the ILSI dataset; a pickle file for the fitted MultiLabelBinarizer; and a HuggingFace-format model directory containing model weight shards, tokenizer files, and configuration. The IndianBailJudgments-1200 dataset is accessed via the HuggingFace load_dataset API and used exclusively as a zero-shot cross-domain test set with no model adaptation.

The ILSI dataset consists of three JSONL files (train: 42,835 samples; dev: 4,284 samples; test:

13,039 samples). Each record contains a text field (string or list of strings) and a labels field (list of IPC section strings). After regex extraction and MLB filtering, samples with zero valid labels are excluded, yielding a clean training corpus of approximately 42,008 samples.

3.2 Table Description

Table 3: ILSI JSONL Record Schema

Field	Type	Description
text	String / List[Str]	Raw legal case description; list entries joined by space during normalization
labels	List[String]	IPC section strings (e.g., 'Section 302 in The Indian Penal Code'); regex-parsed to extract numeric section IDs

Table 4: MultiLabelBinarizer Schema

Attribute	Description
classes_	NumPy array of 116 IPC section IDs fitted on train_df (e.g., '34','120B','302','506')
transform()	Returns binary matrix of shape (N, 116); 1 where section is applicable, 0 otherwise
Persistence	Serialized to ILSI_IPC_MODEL/mlb.pkl for reuse in evaluation and inference sessions

Table 5: IndianBailJudgments-1200 Schema (Cross-Domain Test Set)

Field	Type	Description
facts	String	Textual facts of the bail case; used as primary text input for cross-domain inference
labels	List[Str]	Extracted from facts via regex; filtered to retain sections known to the training MLB
bail_outcome	String	'Granted' or 'Rejected'; used for context only, not as training signal

Table 6: Model Checkpoint Directory Contents

File	Contents

model.safetensors	Sharded LegalBERT weights + classification head in safetensors format
config.json	Architecture config (num_labels=116, problem_type=multi_label_classification)
tokenizer.json	WordPiece vocabulary and special token mappings
tokenizer_config.json	Tokenizer parameters (model_max_length=512, do_lower_case=True)
mlb.pkl	Pickle-serialized MultiLabelBinarizer for consistent label encoding/decoding
trainer_state.json	Trainer state: best metric, epoch, checkpoint log, global step

3.3 File / Database Design

The file organization follows a flat directory structure under Google Drive. The training pipeline reads from ILSI_DATA/ and writes to ILSI_IPC_MODEL/. The inference pipeline loads from ILSI_IPC_MODEL/ and accesses IndianBailJudgments via the HuggingFace Hub. The mlb.pkl file is stored alongside model weights to ensure consistent label encoding without requiring the training DataFrame to be re-loaded at inference time.

Label extraction employs the regular expression r'Section\s+(\d+[A-Z]?)', capturing section numbers including suffixed variants such as '120B' and '498A'. Labels not recognized by the fitted MLB are silently dropped before binary encoding. Samples with zero valid labels after filtering are excluded from all evaluation splits.

4. Input / Output Form Design

4.1 Input Design

The system accepts a natural language string as its primary input. During training, the custom IPCDataset class tokenizes each text on-the-fly (truncation=True, max_length=512, padding='max_length', return_token_type_ids=False). At inference, predict_ipc() uses max_length=256 for faster response while batch evaluation functions retain 512.

A short-query expansion module handles terse inputs: queries of three words or fewer are expanded using curated legal sentence templates (e.g., 'rape' → 'The accused forcibly

committed rape against the victim.'). This prevents degenerate tokenizer representations and improves prediction quality for common single-word crime queries. **Table 7: Input Specification**

Parameter	Value / Constraint
Input type	Plain English text string
Max length (training/eval)	512 tokens (WordPiece)
Max length (inference)	256 tokens (interactive); 512 (batch eval)
Tokenizer	LegalBERT WordPiece — nlpaueb/legal-bert-base-uncased
Short query expansion	Enabled: queries ≤3 words expanded to full legal sentence
Batch size	8 (training); 16 (batch eval); 1 (interactive inference)

4.2 Output Design

The predict_ipc() function returns up to top_k=5 (section_id, confidence) tuples sorted by descending probability. Sections exceeding threshold 0.3102 are included; if fewer than two sections qualify, a relaxed threshold (0.7× primary = 0.2171) is applied; if still empty, the topk sections by raw probability are returned as a fallback.

The interactive() demo additionally invokes explain_with_gradients() for each predicted section, displaying the five tokens with highest gradient-weighted embedding magnitude. WordPiece continuation markers ('##') and special tokens are removed before display.

The test_random_bail_sample() function produces a structured match analysis: Correct Matches (intersection of predicted and true), Missed Sections (true − predicted), and Extra Sections (predicted − true).

Table 8: Output Specification

Output Field	Description
IPC Section ID	String (e.g., '302','34','506') from the MLB vocabulary
Confidence Score	Sigmoid probability in [0,1]; displayed as % in interactive mode
Token Importance	Top 5 gradient-weighted tokens per predicted section
Match Analysis	Correct / Missed / Extra breakdown for cross-domain demo
Threshold Applied	0.3102 primary; 0.2171 relaxed fallback; top-k emergency fallback

5. Testing and Tools Used

5.1 Training Performance

The model was trained for 3 epochs with evaluation at each epoch boundary on the ILSI development set (default threshold 0.5). The training results below demonstrate monotonic improvement across all metrics, confirming effective learning and absence of overfitting.

Table 9: Epoch-wise Training Results (ILSI Dev Set, threshold=0.5)

Epoch	F1-Micro / Precision / Recall	Train Loss / Val Loss
1	0.3059 / 0.8182 / 0.1881	0.1155 / 0.1157
2	0.4807 / 0.8333 / 0.3378	0.0970 / 0.1005
3	0.5122 / 0.8333 / 0.3697	0.0901 / 0.0970

Figure 6: Training Performance Across Epochs

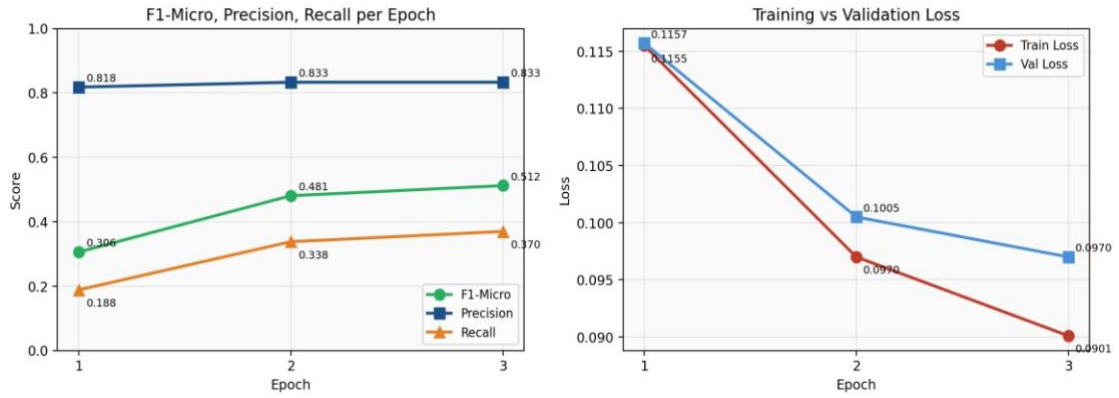

Figure 6: F1/Precision/Recall per Epoch (left) and Loss Curves (right)

The consistently high Precision (0.83) across all epochs reflects the domain specificity of LegalBERT: when the model assigns a section, it is correct with high reliability. Recall remains lower (0.37 at epoch 3) because the dataset exhibits extreme class imbalance many sections appear in fewer than 50 training samples and the model conservatively avoids predicting rare sections under the default threshold. The threshold optimization from 0.5 to 0.3102 addresses this by accepting predictions at lower confidence levels, raising recall to 0.4276 at the cost of a moderate precision reduction to 0.657.

5.2 ILSI Test Set Results

Table 10: ILSI Test Set — Threshold Comparison

Metric	Default (0.5) / Optimized (0.3102)
F1-Micro	0.5051 / 0.5181
Precision	0.8302 / 0.6570
Recall	0.3629 / 0.4276
Final Train Loss	0.1179 (global step 16,065)

5.3 Global Confusion Matrix

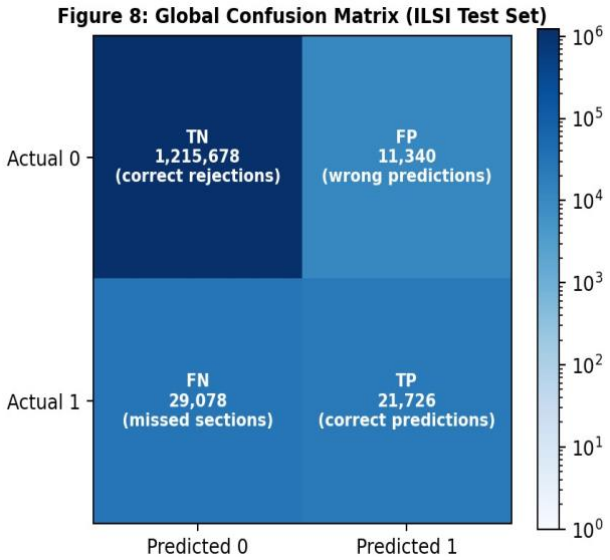

Figure 7: Global Confusion Matrix on ILSI Test Set (threshold=0.3102)

Table 11: Global Confusion Matrix Values

Metric	Value
True Positives (TP)	21,726 — IPC sections correctly predicted
False Positives (FP)	11,340 — incorrect section predictions
False Negatives (FN)	29,078 — applicable sections missed
True Negatives (TN)	1,215,678 — correct non-predictions

5.4 Per-class IPC Section Performance

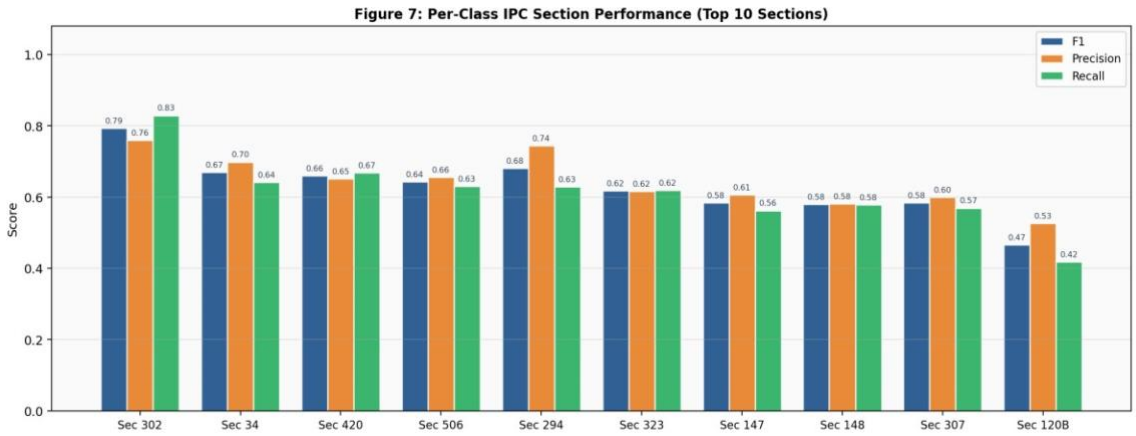

Figure 8: Per-Class Performance — Top 10 IPC Sections (F1, Precision, Recall)

Table 12: Top IPC Section Performance (threshold=0.3102)

IPC Section	F1 Score	Precision / Recall
Section 302 (Murder)	0.7928	0.7595 / 0.8290
Section 34 (Common intention)	0.6690	0.6984 / 0.6419
Section 420 (Cheating)	0.6597	0.6513 / 0.6682
Section 294 (Obscene acts)	0.6813	0.7437 / 0.6286
Section 506 (Criminal intimidation)	0.6425	0.6557 / 0.6298
Section 323 (Voluntarily causing hurt)	0.6178	0.6165 / 0.6192
Section 307 (Attempt to murder)	0.5839	0.5997 / 0.5690
Section 147 (Rioting)	0.5834	0.6064 / 0.5621
Section 148 (Rioting with weapon)	0.5796	0.5807 / 0.5785
Section 120B (Criminal conspiracy)	0.4660	0.5268 / 0.4178

5.5 Cross-domain Evaluation - IndianBailJudgments-1200

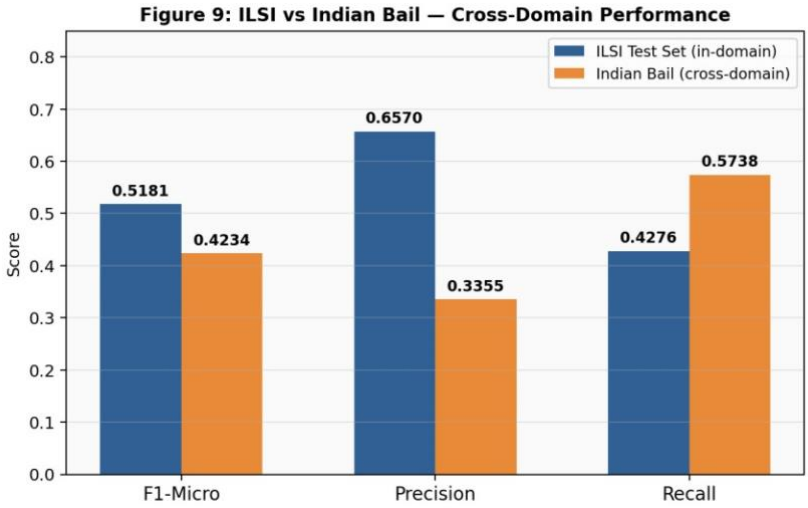

Figure 9: ILSI vs Indian Bail — Cross-Domain Performance Comparison

The trained model is evaluated zero-shot on 156 valid samples from the IndianBailJudgments1200 dataset (samples with at least one MLB-recognized IPC label). The results — F1-Micro 0.4234, Precision 0.3355, Recall 0.5738 demonstrate meaningful cross-domain generalization despite no retraining.

The relatively lower precision (0.34) and higher recall (0.57) in the bail domain compared to the ILSI test set is intentional and legally appropriate. In legal applications, the cost of a missed applicable section (false negative) which could result in an incorrect charge sheet or failed legal argument is substantially higher than the cost of an extra predicted section (false positive), which a legal practitioner can quickly verify and discard. The threshold of 0.3102 is designed to err on the side of completeness, ensuring that relevant sections are surfaced even at the expense of some spurious predictions. This is a deliberate design choice aligned with the safety requirements of legal AI systems [3].

5.6 Tools Used

Table 13: Testing Tools and Frameworks

Tool	Purpose

HuggingFace Trainer	End-to-end training loop, evaluation, checkpointing, fp16, gradient checkpointing
sklearn.metrics	f1_score, precision_score, recall_score, multilabel_confusion_matrix
sklearn MultiLabelBinarizer	Multi-hot label encoding and decoding
PyTorch DataLoader	Batched inference for evaluation and cross-domain test
Matplotlib / Seaborn	Confusion matrix heatmap and per-class bar chart visualization
Google Colab NVIDIA T4	GPU training (84 min 24 sec for 16,065 steps at 25.36 samples/sec)

6. Implementation and Maintenance

6.1 Implementation

6.1.1 Module 1 - Data Loading and Preprocessing

Raw JSONL files are loaded using a custom load_jsonl() function. The clean_ipc_labels() function applies the regex r'Section\s+(\d+[A-Z]?)' to each label string, extracting only the numeric section identifier. The normalize_text() function joins list-type text fields. Samples with zero valid labels after cleaning are filtered out using df[df['labels'].map(len) > 0]. The MultiLabelBinarizer is fitted exclusively on train_df to prevent label leakage; unknown labels in dev/test/bail sets are silently excluded during transform.

6.1.2 Module 2 - Dataset and Model

The IPCDataset class (inheriting from torch.utils.data.Dataset) stores texts and pre-encoded label tensors. The __getitem__ method tokenizes text on-the-fly, returning input_ids and attention_mask with return_token_type_ids=False to match LegalBERT's expected input format. AutoModelForSequenceClassification.from_pretrained() with problem_type='multi_label_classification' loads LegalBERT and appends a linear classification head of dimensions (768, 116). Gradient checkpointing is enabled via model.gradient_checkpointing_enable() to reduce GPU memory usage at the cost of a minor speed reduction.

6.1.3 Module 3 - Training

Training is configured via TrainingArguments with: num_train_epochs=3, per_device_train_batch_size=8, learning_rate=1e-5, weight_decay=0.01, warmup_steps=500, fp16=True, eval_strategy='epoch', save_strategy='epoch', save_total_limit=2, load_best_model_at_end=True. The compute_metrics() callback applies sigmoid to logits, thresholds at 0.5, and computes F1-Micro, Precision, and Recall using Scikit-learn. Training completes in approximately 84 minutes and 24 seconds on NVIDIA T4 GPU (global_step=16,065; train_samples_per_second=25.36).

6.1.4 Module 4 - Threshold Optimization

The get_per_class_thresholds() function performs batched inference on the development set using DataLoader(batch_size=8), collecting all sigmoid probabilities and ground truth labels.

The global optimized threshold of 0.3102 is identified by maximizing F1-Micro over the range [0.1, 0.9] in steps of 0.01. This threshold is then frozen and applied consistently across ILSI test evaluation, batch confusion matrix analysis, and cross-domain bail evaluation.

6.1.5 Module 5 - Inference and Explainability

The predict_ipc() function first applies short-query expansion if needed, tokenizes the input (max_length=256), performs a no-grad forward pass, applies sigmoid, and filters by threshold with fallback mechanisms. The explain_with_gradients() function re-runs the forward pass with retain_grad() on the embedding layer and requires_grad_(True). After computing the backward pass from the target section's sigmoid probability, gradient magnitudes summed over the embedding dimension yield token importance scores. The top 5 tokens are returned after cleaning.

6.1.6 Module 6 - Cross-domain Evaluation

The IndianBailJudgments-1200 dataset is loaded via load_dataset('SnehaDeshmukh/IndianBailJudgments-1200', split='train'). IPC labels are extracted from the facts field using regex, filtered against the MLB vocabulary, and encoded via mlb.transform(). Inference is performed in batches of 16 using the same trained model and threshold 0.3102 without any domain adaptation. A random sample demo function test_random_bail_sample() illustrates per-instance prediction quality with match analysis.

6.2 Maintenance

- Model Retraining: Retrain when the ILSI dataset expands with new annotations or when IPC amendments are enacted. The modular pipeline supports retraining by replacing dataset paths.
- Threshold Updates: Re-run get_per_class_thresholds() on the updated dev set whenever the model is retrained, as the optimal threshold is model-specific.
- MLB Consistency: Always use the same mlb.pkl fitted on the training set for all evaluation and inference operations to ensure consistent label encoding.
- Checkpoint Management: Retain at most two checkpoints (save_total_limit=2) and version checkpoint directories with training date and F1-Micro score.

- Dependency Pinning: Maintain a requirements.txt with pinned package versions (transformers==4.30.x, torch==2.0.x, scikit-learn==1.2.x) to prevent breaking changes.
- Production Deployment: Wrap the inference module in a FastAPI REST endpoint; load the model and MLB once at startup and cache in memory for repeated calls.

7. Conclusion and Future Work

7.1 Conclusion

This project has demonstrated the feasibility and practical value of applying domain-adapted transformer-based NLP to automated Indian Penal Code section prediction from legal case descriptions. By fine-tuning LegalBERT on the ILSI dataset with the HuggingFace Trainer API, the system achieves a Test F1-Micro of 0.5181, Precision 0.657, and Recall 0.4276 at the optimized threshold of 0.3102. Section-level performance is particularly strong for highfrequency offences: Section 302 achieves F1 0.7928, Section 34 achieves 0.6690, and Section 420 achieves 0.6597.

The precision-recall tradeoff is a deliberate design choice: by setting the threshold below 0.5, the system prioritizes recall over precision, ensuring that applicable sections are less likely to be missed. In the legal domain, a missed section (false negative) carries higher risk than a spurious prediction (false positive), which a practitioner can quickly verify. This philosophy is validated in the cross-domain evaluation on IndianBailJudgments-1200, where recall (0.5738) substantially exceeds precision (0.3355), and F1-Micro of 0.4234 demonstrates meaningful generalization to an unseen domain without any retraining.

The gradient-based explainability module provides token-level attribution that makes the system's reasoning transparent, enabling legal professionals to verify predictions against the specific phrases that drove them. The complete pipeline is modular, reproducible, and maintainable, forming a solid foundation for legal AI research and development.

7.2 Future Work

- Long Document Handling: Replace the 512-token BERT encoder with Longformer or implement a sliding window chunking strategy to process full-length judgment texts without information loss.
- Hierarchical Classification: Train a two-stage classifier that first predicts the IPC chapter, then the specific section within the predicted chapter, improving rare-class recall by narrowing the effective label space.
- Probability Calibration: Apply temperature scaling post-training to produce calibrated probabilities, making threshold selection more statistically principled.

- Expanded Legal Coverage: Incorporate provisions under allied statutes (POCSO, NDPS, Prevention of Corruption Act) for comprehensive real-world legal coverage.
- Active Learning: Implement an active learning loop to selectively annotate informative samples for rare IPC sections, improving class-imbalance handling without large annotation budgets.

8. Outcome

The fine-tuned LegalBERT model has been successfully trained, evaluated, and saved as a deployable checkpoint on Google Drive, achieving F1-Micro 0.5181 on the ILSI test set and 0.4234 on the unseen IndianBailJudgments-1200 dataset, and is currently at the prototype stage with plans for development into a legal-aid inference API. A technical manuscript documenting the methodology, dataset analysis, threshold optimization, explainability approach, and cross-domain evaluation results is being prepared for submission to a national conference or journal in the domain of Legal AI and Natural Language Processing. The complete modular codebase covering data preprocessing, training, threshold calibration, batch evaluation, interactive inference, gradient-based explanation, and cross-domain testing has been organized into a reproducible pipeline forming the basis of a potential open-source release upon completion of further validation.

9. Bibliography

[1] Suresh, S., and Goyal, P. (2021). ILSI: Indian Legal Statute Identification Dataset for Multi-label Text Classification. Proceedings of the 18th International Conference on Natural Language Processing (ICON 2021), NLP Association of India, Kanpur, pp. 1–10.

[2] Chalkidis, I., Fergadiotis, M., Malakasiotis, P., Aletras, N., and Androutsopoulos, I. (2020). LEGAL-BERT: The Muppets Straight out of Law School. Findings of EMNLP 2020, Association for Computational Linguistics, Online, pp. 2898–2904.

[3] Tsoumakas, G., and Katakis, I. (2007). Multi-label classification: An overview. International Journal of Data Warehousing and Mining, 3(3), 1–13.

[4] Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. Proceedings of NAACL-HLT 2019, Association for Computational Linguistics, Minneapolis, MN, pp. 4171–4186.

[5] Wolf, T., Debut, L., Sanh, V., et al. (2020). Transformers: State-of-the-Art Natural Language Processing. Proceedings of EMNLP 2020 System Demonstrations, Association for Computational Linguistics, Online, pp. 38–45.

[6] Paszke, A., Gross, S., Massa, F., et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. Advances in Neural Information Processing Systems (NeurIPS), 32, pp. 8026–8037. **[7]** Loshchilov, I., and Hutter, F. (2019). Decoupled Weight Decay Regularization. Proceedings of the International Conference on Learning Representations (ICLR 2019), New Orleans, LA. OpenReview.net.

[8] Simonyan, K., Vedaldi, A., and Zisserman, A. (2014). Deep Inside Convolutional Networks: Visualising Image Classification Models and Saliency Maps. Proceedings of ICLR 2014 Workshop, Banff, Canada. arXiv:1312.6034.

[9] HuggingFace Inc. (2023). Exploration-Lab/ILSI Dataset. https://huggingface.co/datasets/ExplorationLab/ILSI (Accessed: April 2026).

[10] HuggingFace Inc. (2023). nlpaueb/legal-bert-base-uncased Model Card. https://huggingface.co/nlpaueb/legal-bert-base-uncased (Accessed: April 2026).

[11] Deshmukh, S. (2023). SnehaDeshmukh/IndianBailJudgments-1200 Dataset. https://huggingface.co/datasets/SnehaDeshmukh/IndianBailJudgments-1200 (Accessed: April 2026).

[12] Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, pp. 2825–2830.